

## # High-speed bottle filling conveyor systems: a C++ engineering reference

**\*\*No single filling machine on Earth achieves 5,000 bottles per minute.\*\*** The fastest documented single-line system — the Krones Modulfill Dual at 100,000 bottles/hour (~1,667 BPM) — uses dual carousels to reach that throughput. [Krones](https://www.krones.com/en/company/press/magazine/reference/ergobloc-l-quantum-leap-in-block-technology.php) [Krones](https://www.krones.com/en/products/machines/modulfill-dual.php) Reaching 5,000 BPM (300,000 BPH) requires **\*\*three to five parallel high-speed lines\*\***, each controlled independently but coordinated through a supervisory system. This reality fundamentally shapes the hardware platform, software architecture, and real-time strategy for production-grade C++ code targeting this throughput class. The Siemens S7-1500 + PROFINET ecosystem dominates commercial bottling (driven by Krones and KHS), while Beckhoff TwinCAT + EtherCAT is gaining ground for motion-intensive applications with its **\*\*50 µs cycle times\*\*** [PLC Programming IO] (https://plcprogramming.io/blog/beckhoff-twincat-programming-tutorial) and native C++ support — making it the strongest platform choice for custom C++ development. Every architectural decision in this domain flows from one constraint: at 5,000 BPM, each bottle gets exactly **\*\*12 milliseconds\*\*** of processing time, and a single missed valve closure wastes product, violates FDA regulations, or crashes the line.

---

## ## 1. The hardware stack powering real-world bottling lines

### ### Control platforms and the OEM ecosystem

The two dominant control architectures in commercial high-speed bottling are Siemens-based and Beckhoff-based, each with distinct strengths.

**\*\*Siemens S7-1500 ecosystem\*\*** is the industry incumbent. Krones — the world's largest bottling OEM — standardizes on S7-1500 CPUs programmed through TIA Portal. Krones Academy explicitly offers training courses titled "Siemens SIMATIC S7-1500 for KRONES machines." [Krones] (https://shop.krones.com/shop/us/en/Academy/Automation/c/acAutomation) The S7-1500T (Technology CPU) handles electronic camming and multi-axis servo synchronization natively. Safety is integrated via F-CPU variants (S7-1517F, S7-1518F) certified to **\*\*SIL 3 / PLe\*\***. [Industrialautomationco] (https://industrialautomationco.com/blogs/news/siemens-s7-1500-advanced-features-explained) Field I/O uses PROFINET-connected ET200SP distributed terminals. Servo drives are SINAMICS S210 [Packagingstrategies] (https://www.packagingstrategies.com/articles/95251-producing-packaging-with-servo-drive-systems) (single-axis packaging) or S120 (multi-axis complex lines), and conveyor VFDs are SINAMICS G120. HMI runs on zenOn (COPA-DATA) touchscreens. The minimum PLC cycle time is **\*\*1 ms\*\*** with PROFINET IRT providing **\*\*250 µs\*\*** update rates for motion control. [Profinet](https://us.profinet.com/profinet-irt-isochronous-real-time/)

**\*\*Beckhoff TwinCAT 3 ecosystem\*\*** is the emerging challenger for high-speed packaging. Tetra Pak uses two Beckhoff C6032 ultra-compact IPCs running TwinCAT to synchronize **\*\*120+ motion axes\*\*** on their Cap Applicator 40 Speed Hyper. [Packaging World](https://www.packworld.com/trends/controls-machine-components/article/22954796/beckhoff-xts-linear-transport-system-synchronizes-120-axes-for-ultrafast-capping-by-tetra-paks-new-magic-machine) ProMach's Pace division uses Beckhoff C6025 IPCs for robotic unscrambling.

[Packaging World](https://www.packworld.com/trends/controls-machine-components/article/22933118/beckhoff-automation-unscrambles-bottling-chaos-in-new-pro-mach-pace-packaging-versasort) TwinCAT runs a real-time kernel on dedicated CPU cores alongside Windows, achieving a minimum base time of **50 μs** with sub-microsecond jitter. [PLC Programming IO](https://plcprogramming.io/blog/beckhoff-twincat-programming-tutorial) EtherCAT delivers **50–100 μs** cycle times [Wikipedia](https://en.wikipedia.org/wiki/EtherCAT) with Distributed Clocks synchronization [PLC Programming IO](https://plcprogramming.io/blog/beckhoff-twincat-programming-tutorial) accurate to **±20 ns**. [TI](https://www.ti.com/content/dam/videos/external-videos/en-us/7/3816841626001/6174375373001.mp4/subassets/f2838x\_ethercat\_distributed\_clocks.pdf) AX8000 multi-axis servo drives and AM8000 servomotors communicate over EtherCAT, [Packaging World](https://www.packworld.com/trends/controls-machine-components/article/22933118/beckhoff-automation-unscrambles-bottling-chaos-in-new-pro-mach-pace-packaging-versasort) with TwinSAFE providing integrated functional safety (SIL 3 / PLe) on the same network. Critically for this project, TwinCAT 3 natively supports **C++** modules via the TcCOM framework, [Beckhoff +2](https://m.beckhoff.com/english/press/pr1613.htm) making it the most natural platform for production C++ code.

**Allen-Bradley ControlLogix/GuardLogix** dominates North American installations. These PLCs run VxWorks internally, use EtherNet/IP with CIP Motion for servo control, and CIP Safety for functional safety. Kinetix 5700 servo drives provide multi-axis capability. The GuardLogix 5580 is certified to **SIL 3 / PLe / Cat 4**. [Automation Ready Panels](https://www.automationreadypanels.com/plc-systems/guardlogix-compactlogix-pld-safety-system/) [Rexelusa](https://pages.rexelusa.com/blog/automation/safe-stopping-options)

**ARM-based controllers (STM32, TI AM6x)** do not appear at the main PLC level in any documented commercial bottling system. They exist at the valve level — Krones' PFR valve contains decentralized embedded electronics [Krones](https://www.krones.com/en/products/machines/modulfill-vfs-m-with-pfr-valves.php) (likely a microcontroller driving a stepper motor and reading a flow meter) — and in EtherCAT/PROFINET slave ASICs. For a C++ implementation, ARM targets are appropriate for sensor nodes, valve-level controllers, or edge computing, but never for primary line control.

### The control hierarchy

A production 5,000 BPM system uses a five-level architecture:

| Level | Function                              | Hardware                                                     | Protocol                            |
|-------|---------------------------------------|--------------------------------------------------------------|-------------------------------------|
| 4     | MES/ERP integration                   | Server / cloud                                               | OPC UA, SQL, MQTT                   |
| 3     | Supervisory / SCADA / line management | Industrial PC                                                | OPC UA, Weihenstephan WS            |
| 2     | Primary control (PLC / IPC)           | S7-1500F or Beckhoff C6030                                   | PROFINET IRT or EtherCAT            |
| 1     | Drive / distributed I/O               | SINAMICS S210/S120 or AX8000, ET200SP or EK1100              | Same fieldbus                       |
| 0     | Sensors, valves, actuators, encoders  | SICK/Keyence photoelectric, E+H flow meters, Baumer encoders | Hardwired analog/digital or IO-Link |

### Sensors, actuators, and filling valves

**Flow meters** are the primary fill measurement for water. Endress+Hauser Promag electromagnetic flow meters (requiring conductivity >20 μS/cm — water qualifies) deliver **±0.2%** accuracy with 150 Hz coil excitation enabling fills

in approximately 1 second. Krones uses inductive flow meters per valve on their Modulfill VFJ/VFS series. [Krones] (<https://www.krones.com/en/products/machines/modulfill-vfs-water.php>) For non-conductive or high-value products, Coriolis mass flow meters (E+H Promass) achieve  $\pm 0.1\%$  but are slower. [Food Engineering] (<https://www.foodengineeringmag.com/articles/96920-how-to-determine-the-best-sensor-for-filling-applications>)

**Filling valves** have evolved from mechanical cam-controlled to fully electronic. The Krones PFR (Proportional Flow Regulator) represents the current state of the art: an electronic controller with integrated stepper motor provides infinitely variable flow speed, [Krones +2] (<https://www.krones.com/en/products/machines/modulfill-vfs-m-with-pfr-valves.php>) eliminating compressed air at the valve and increasing output by **25%** over conventional two-speed valves. [Krones] (<https://www.krones.com/en/products/machines/modulfill-vfs-m-with-pfr-valves.php>) [Krones] (<https://www.krones.com/en/products/machines/modulfill-vfs-water.php>) Conventional systems use electropneumatic valves from Festo or SMC with 2–5 ms full-stroke response times.

**Servo drives and motors** for star wheels and carousels include Siemens SINAMICS S210 paired with 1FK2 SIMOTICS servomotors [Packagingstrategies] (<https://www.packagingstrategies.com/articles/95251-producing-packaging-with-servo-drive-systems>) (PROFINET, One Cable Connection), Beckhoff AX8000 with AM8000 servomotors (EtherCAT), [iebmedia] (<https://iebmedia.com/technology/industrial-ethernet/flexible-control-technology-with-fast-cycle-times-increases-efficiency/>) and B&R ACOPOS P3 (POWERLINK). Conveyor belt drives use VFDs — SINAMICS G120 (0.55–250 kW) is the workhorse for Krones lines, with SEW DRE motors documented on KHS systems.

**Photoelectric sensors** for bottle detection come from SICK, Keyence, Banner, and IFM, with response times under 100  $\mu$ s for no-bottle/no-fill interlocking. **Encoders** (Heidenhain, Baumer, SICK) provide 21-bit absolute resolution (2,097,152 increments/revolution) for carousel position tracking. **Load cells** (Mettler Toledo, HBM) provide  $\pm 0.1\%$  accuracy for weight-based filling systems [HBM] (<https://www.hbm.com/en/6992/using-weighing-technology-for-automated-filling-processes/>) [HBK] (<https://www.hbkworld.com/en/solutions/applications/weighing/process-weighing/weighing-technology-for-automated-filling-processes>) like the Krones Modulfill NWJ. **Inspection systems** from Krones Checkmat and Heuft verify fill level, cap closure, and label placement downstream.

### Real-world OEM systems and their limits

| OEM / System                  | Max Documented Speed    | Filling Heads               | Key Technology                               |
|-------------------------------|-------------------------|-----------------------------|----------------------------------------------|
| --- --- --- ---               |                         |                             |                                              |
| Krones Modulfill Dual         | 100,000 BPH (1,667 BPM) | ~120–140 across 2 carousels | PFR valves, flow meters, dual-carousel split |
| Krones Modulfill VFS-C (cans) | 135,000 CPH (2,250 CPM) | 28–182                      | PFR valves                                   |
| KHS Innofill PET DRV          | 90,000 BPH (1,500 BPM)  | —                           | CFD-optimized, 3-speed pneumatic             |
| Sidel Super Combi             | 60,000 BPH (1,000 BPM)  | —                           | 5-in-1 integrated block                      |
| Fogg Filler F12               | 72,000 BPH (1,200 BPM)  | Up to 120                   | Gravity, HEPA enclosure                      |

To reach **300,000 BPH (5,000 BPM)**, three Krones 100K ErgoBloc L lines operating in parallel is the most realistic architecture. Each line spans approximately 40–60 meters from blow molding through palletizing. The Krones Modulfill uses a **4,320 mm pitch diameter** carousel with PFR valves [Krones] (<https://www.krones.com/en/products/machines/modulfill-vfs-water.php>) filling at up to **250 mL/sec** for water. [Krones] (<https://www.krones.com/en/products/machines/modulfill-vfs-m-with-pfr-valves.php>) [Krones]

(<https://www.krones.com/en/products/machines/modulfill-vfs-water.php>) A rule-of-thumb for still water at 0.5 L is approximately 700 bottles per valve per hour, meaning 300,000 BPH would require approximately **\*\*430 filling valves\*\*** total across all parallel lines.

---

## ## 2. Real-time timing budget for 12 ms per bottle

At 5,000 bottles per minute, the fundamental timing constraint is **\*\*12 ms per bottle\*\*** ( $60,000 \text{ ms} \div 5,000$ ). Every control decision, valve actuation, and position correction must complete within this window. This is exclusively achievable with rotary fillers [Filling Equipment](<https://www.fillingequipment.com/product/rotary-water-filling-machine/>) – inline/linear systems top out at 120–300 BPM. [Acasi](<https://acasi.com/collections/filling-machines>)

### ### Rotary filler timing analysis

For a single 120-head rotary carousel targeting 1,667 BPM (one of three parallel lines), the carousel rotates at approximately **\*\*13.9 RPM\*\*** (one rotation every 4.32 seconds). With a 270° filling arc, each head has approximately **\*\*3.24 seconds\*\*** of available fill time – comfortable for 0.5 L water at 250 mL/sec. Transport and star-wheel transfer consume roughly 45–60° of rotation (approximately 720–960 ms). The critical timing occurs at handoff points where servo-driven star wheels must synchronize bottle position to  $\pm 0.1 \text{ mm}$  accuracy.

### ### Jitter and determinism requirements

The timing hierarchy breaks into three tiers with progressively tighter constraints:

**\*\*Motion control (servo synchronization)\*\*** demands the tightest tolerances. Multi-axis coordination of carousel drive, infeed star wheel, discharge star wheel, and capper requires **\*\*<1  $\mu\text{s}$  jitter\*\***. EtherCAT Distributed Clocks deliver [PLC Programming IO](<https://plcprogramming.io/blog/ethercat-protocol-tutorial-complete-guide>)  $\pm 20 \text{ ns}$  synchronization [TI]([https://www.ti.com/content/dam/videos/external-videos/en-us/7/3816841626001/6174375373001.mp4/subassets/f2838x\\_ethercat\\_distributed\\_clocks.pdf](https://www.ti.com/content/dam/videos/external-videos/en-us/7/3816841626001/6174375373001.mp4/subassets/f2838x_ethercat_distributed_clocks.pdf)) – far exceeding this requirement. PROFINET IRT achieves <1  $\mu\text{s}$  with Conformance Class C devices. [Profinet](<https://us.profinet.com/wp-content/uploads/2020/10/PROFINET-IRT-final.pdf>) A typical high-speed bottling line synchronizes **\*\*8–20 servo axes\*\*** phase-locked to a single master encoder on the main carousel shaft.

**\*\*Fill control (valve timing)\*\*** requires **\*\*<100  $\mu\text{s}$  jitter\*\***. The flow meter signals target volume reached, and the PFR valve must close [Krones](<https://www.krones.com/en/products/machines/modulfill-vfs-water.php>) within **\*\*<1 ms\*\*** to maintain fill accuracy within  $\pm 0.3\%$  of target volume. Late valve closure by even 1 ms at 250 mL/sec means 0.25 mL overfill – which compounds across hundreds of thousands of bottles into significant product waste and potential regulatory violations.

**\*\*Sequence logic and supervisory\*\*** operates at 1–20 ms cycle times with millisecond-range jitter tolerance. PackML state transitions, recipe management, and HMI updates fall in this tier.

### ### RTOS selection matrix

| Platform                           | Cycle Time                    | Best For                               | Real-World Usage                                  |
|------------------------------------|-------------------------------|----------------------------------------|---------------------------------------------------|
| ---                                | ---                           | ---                                    | ---                                               |
| **Beckhoff TwinCAT 3**             | 50 µs minimum                 | Primary control with C++               | Tetra Pak (120+ axes), ProMach                    |
| **Siemens S7-1500 + PROFINET IRT** | 250 µs–1 ms                   | PLC-centric                            | Krones/KHS lines   Krones Modulfill, KHS Innofill |
| **VxWorks**                        | Microsecond-level IRQ latency | Inside Rockwell PLCs                   | Allen-Bradley ControlLogix/CompactLogix           |
| **QNX**                            | Microsecond-level, SIL 3 cert | Safety-critical subsystems             | Robotics, medical, industrial embedded            |
| **FreeRTOS on ARM**                | 1 ms tick (configurable)      | Valve-level controllers, sensor nodes  | Embedded sub-systems                              |
| **Linux PREEMPT_RT**               | 125 µs demonstrated           | Soft RT: HMI, SCADA, data logging      | Supervisory stations                              |
| **Bare-metal ARM**                 | Zero overhead, immediate ISR  | High-speed counters, safety interlocks | Encoder tracking, analog measurement              |

For a custom C++ implementation, **Beckhoff TwinCAT 3 on a C6030/C6032 IPC** is the optimal primary platform. [iebmedia](https://iebmedia.com/technology/industrial-ethernet/flexible-control-technology-with-fast-cycle-times-increases-efficiency/) It provides native C++ execution in hard real-time (TcCOM modules), EtherCAT for deterministic I/O, and the lowest achievable cycle times (50 µs). [Beckhoff](https://infosys.beckhoff.com/content/1033/tcsystemmanager/1086417035.html) The internal servo current/torque loop runs at **31.25–62.5 µs** inside the drive, independent of the fieldbus. The position loop synchronizes to the EtherCAT cycle at **125 µs–1 ms** using Cyclic Synchronous Position (CSP) mode.

### Task priority architecture

| Priority             | Task                             | Cycle Time  | Notes                                   |
|----------------------|----------------------------------|-------------|-----------------------------------------|
| ---                  | ---                              | ---         | ---                                     |
| **Hardware IRQ**     | E-stop, safety interlocks        | <10 µs      | Bare-metal ISR, independent of PLC scan |
| **Hardware counter** | Encoder counting                 | Continuous  | Dedicated hardware, independent of CPU  |
| **RT Task 1**        | Motion control / servo sync      | 50–250 µs   | EtherCAT DC synchronized                |
| **RT Task 2**        | Fill control (valve timing, PID) | 250 µs–1 ms | Flow meter read → PFR actuation         |
| **RT Task 3**        | Sequence logic / PackML          | 1–4 ms      | Main program cycle                      |
| **Background**       | OPC UA, SCADA, HMI               | 10–100 ms   | Non-deterministic communication         |
| **Lowest**           | Diagnostics, logging, trends     | 100 ms–1 s  | Data historian                          |

### 3. Software architecture: where C++ meets IEC 61131-3

#### The hybrid approach used in industry

Production bottling lines never use pure C++ or pure IEC 61131-3 — they use both. **IEC 61131-3 Structured Text (ST)** handles machine sequencing, Boolean logic, and maintenance-friendly troubleshooting [Roboticsys](https://www.roboticsys.com/blog/general-purpose-programming-languages-are-the-future-for-the-industrial-automation-industry) — a floor technician can read ladder logic but not templated C++. **C++ handles complex algorithms, proprietary IP, motion control math, vision integration, and database interfacing** — areas where ST's limitations become painful. [Roboticsys](https://www.roboticsys.com/blog/general-purpose-programming-languages-

are-the-future-for-the-industrial-automation-industry) The Beckhoff TcCOM framework makes this seamless: C++ modules and IEC 61131-3 function blocks communicate through shared interfaces in the same real-time context. [Beckhoff](https://m.beckhoff.com/english/press/pr1613.htm)

### ### TcCOM: Beckhoff's C++ real-time framework

TwinCAT Component Object Model (TcCOM) extends Microsoft COM for real-time execution. [beckhoff](https://infosys.beckhoff.com/content/1033/tc3\_c/2491035531.html) Each C++ module is a self-contained unit compiled with MSVC, described by a TMC (TwinCAT Module Class) file, and instantiated by the TwinCAT ObjectServer. [beckhoff](https://infosys.beckhoff.com/content/1033/tc3\_c/2491035531.html) Modules implement mandatory interface `ITComObject`` [Beckhoff](https://infosys.beckhoff.com/content/1033/tc3\_c/2491035531.html) plus optional interfaces: `ITcCyclic`` for cyclic execution [Beckhoff](https://infosys.beckhoff.com/content/1033/tc3\_c/2491035531.html) (registered with a TwinCAT task at a specific priority), `ITcADI`` for data area access, and `ITcTask`` for task timing information. [beckhoff](https://infosys.beckhoff.com/content/1033/tc3\_c/2491035531.html) Modules follow a state machine (Init → Pre-Op → Safe-Op → Op → Error) and are reference-counted. Inter-module communication uses interface queries through the ObjectServer, [Beckhoff](https://infosys.beckhoff.com/content/1033/tc3\_c/2491035531.html) enabling C++ modules to call PLC function block methods and vice versa. [beckhoff](https://infosys.beckhoff.com/content/1033/tc3\_c/2491035531.html) Siemens offers similar capability through ODK 1500S (Open Development Kit), which embeds C/C++ code as S7 blocks within TIA Portal. [Siemens](https://assets.new.siemens.com/siemens/assets/api/uuid:f088231a33215e94cb4a9a0f39cd56dadfa765b0/dffa-b10006-03-7600-ws-simatic-s7-1500-software-controller-en.pdf) B&R Automation Studio supports ANSI C/C++ alongside IEC 61131-3 with shared data types. [B&R Industrial Automation](https://www.br-automation.com/en/products/software/additional-information/programming/)

### ### PackML state machine implementation

The ISA-TR88.00.02-2022 PackML standard defines **17 states** that every packaging machine must implement. [Wikipedia](https://en.wikipedia.org/wiki/PackML) [Schneider Electric](https://product-help.schneider-electric.com/Machine%20Expert/V1.1/en/PackMLli/PackMLli/PackMLLb\_Overview/PackMLLb\_Overview-3.htm) The states divide into **Wait States** (machine holds until commanded: Stopped, Idle, Suspended, Held, Complete, Aborted) and **Acting States** (machine actively transitioning: Starting, Execute, Holding, Unholding, Suspending, Unsuspending, Completing, Stopping, Aborting, Clearing, Resetting). [GitHub](https://github.com/hsu-aut/IndustrialStandard-ODP-PackML) Two transitions deserve special attention: **Aborting** can be entered from any state [ISA Interchange](https://blog.isa.org/reaching-the-next-industrial-level-with-packaging-machine-communication-standards) (triggered by E-stop or critical fault), [Cospa](https://www.cospa.ru/upload/iblock/2e5/YASKAWA-Designing-your-first-PackML-implementation-for-machine-control-WP.MTN.02.pdf) and **Held** versus **Suspended** distinguishes internal disturbances (e.g., filler jam) from external disturbances (e.g., upstream blow molder fault). [GitHub](https://github.com/hsu-aut/IndustrialStandard-ODP-PackML)

In C++, this maps to a finite state machine with enum states and a transition table. The State design pattern provides clean encapsulation:

```
```cpp
```

```
enum class PackMLState : uint8_t {
    Stopped, Starting, Idle, Suspended, Execute,
    Holding, Held, Unholding, Completing, Complete,
    Aborting, Aborted, Clearing, Stopping,
    Resetting, Suspending, Unsuspending
};
```

```
class IMachineState {
public:
    virtual void enter(MachineContext& ctx) = 0;
    virtual void execute(MachineContext& ctx) = 0;
    virtual void exit(MachineContext& ctx) = 0;
    virtual PackMLState id() const = 0;
    virtual ~IMachineState() = default;
};
...
```

PackTags — standardized data tags for machine communication — fall into three categories: **Command** (Start, Stop, Reset, Abort, Hold, Unhold, Suspend, Unsuspend, Clear, Complete), **Status** (StateCurrent, MachSpeed, CurMachSpeed, EquipmentInterlock, ProductionCounters), and **Admin** (ProdConsumedCount, ProdProcessedCount, ProdDefectiveCount, MachDesignSpeed, StopReason). [Wikipedia +2](<https://en.wikipedia.org/wiki/PackML>) These feed OEE calculations directly: [En Academic](<https://en-academic.com/dic.nsf/enwiki/11568982>) Availability from time-in-state tracking, Performance from CurMachSpeed/MachDesignSpeed, Quality from processed versus defective counts. [Cospa](<https://www.cospa.ru/upload/iblock/2e5/YASKAWA-Designing-your-first-PackML-implementation-for-machine-control-WP.MTN.02.pdf>)

### ### PID control for fill volume accuracy

For water filling with electromagnetic flow meters, the PID controller operates in a **cascade configuration**: the outer loop controls fill volume (setpoint = target volume, PV = flow meter totalizer), and the inner loop controls flow rate (setpoint from outer loop, PV = instantaneous flow rate). The inner loop rejects supply pressure disturbances. Key implementation techniques:

- **Anti-windup**: Clamp the integral term when the output saturates (valve fully open or closed). Back-calculation method is preferred — subtract the saturated output from the actual output and feed it back to the integrator with a tracking time constant.
- **Feedforward**: Pre-set the valve position from known recipe parameters (bottle size, product density, supply pressure). PID corrects only the residual error, dramatically reducing settling time.
- **Two-speed fill profile**: Fast fill to ~90% volume at maximum flow rate, then slow fill for the remaining 10% to prevent overshoot and foaming. The PFR valve's infinitely variable speed makes this seamless. [Krones](<https://www.krones.com/en/products/machines/modulfill-vfs-m-with-pfr-valves.php>)
- **Drip compensation**: After valve closure, product remaining in the nozzle drips into the bottle. The controller learns this residual volume and closes the valve early by exactly that amount. Endress+Hauser's Dosimag/Dosimass flow meters include an integrated batching algorithm that handles this autonomously. [Endress+Hauser]

(<https://www.us.endress.com/en/endress-hauser-group/Case-studies-application-notes/RONCHI-filling-machine-with-Dosimag>)

Target accuracy for water bottling is  $\pm 0.25\%$  to  $\pm 0.5\%$  standard deviation with electromagnetic flow meters, [DTM] (<https://dtmpackaging.com/machines/filler/purefil-in-line-fillers/>) or  $\pm 0.1\%$  with load-cell-based weighing systems. [Filling Insider] (<https://www.fillinginsider.com/a-guide-to-liquid-filling-technologies-exploring-weight-gravity-flow-meter-volumetric-and-piston-fillers/>) [HBK] (<https://www.hbkworld.com/en/solutions/applications/weighing/process-weighing/weighing-technology-for-automated-filling-processes>) For a 500 mL bottle, this means  $\pm 1.25$ – $2.5$  mL variation – well within the EU e-mark tolerable negative error of 15 mL.

### ### Reject and divert logic

Bad fills are detected by in-filler flow meter totalizer comparison (immediate), downstream checkweighers (Mettler-Toledo, Wipotec – up to 1,200+ packages/min per lane,  $\pm 0.1$  g accuracy), and vision systems (fill level, cap presence, label placement). The challenge is that detection and rejection stations are physically separated by  $0.5$ – $3$  meters of conveyor.

Two tracking approaches are used. **Bit-shift registers** are driven by encoder pulses: each bit represents a physical position on the conveyor, and a reject flag shifts through the register as the conveyor moves. When the flag reaches the bit position corresponding to the reject station, the mechanism fires. [PLCS.net] (<https://www.plctalk.net/forums/threads/advice-tracking-a-reject-on-a-conveyor.124552/>) [PLCS.net] (<https://www.plctalk.net/qanda/archive/index.php/t-87892.html>) This provides the tightest position tracking for fixed-pitch applications. **FIFO queues** load bottle status when a photoeye detects entry and unload when a photoeye at the reject station detects arrival [Solis PLC] (<https://www.solisplc.com/tutorials/understanding-and-using-fifos-in-studio-5000>) – more robust for variable spacing. [MrPLC.com] (<https://mrplc.com/files/file/601-conveyor-fifo-first-in-first-out-wreject/>) Reject mechanisms include air blast (fastest, no moving parts, preferred for lightweight PET at high speed), diverter arms, push cylinders, [Acefilling] (<https://www.acefilling.com/blog/precision-control-why-it-matters-in-liquid-filling-machine-performance>) [Neminc] (<https://neminc.com/machines/inspect-reject-systems/inspection-system/>) and the KHS Innocheck GRS servo-matched ejection fingers rated at  $72,000$  bottles/hour without compressed air. [Khs] (<https://www.khs.com/en/products/machines-equipment/detail/innocheck-grs-rejection-system>)

---

## ## 4. Conveyor control: tracking bottles at 6.67 meters per second

### ### Encoder-based position tracking

At 5,000 BPM with 80 mm bottle pitch, the conveyor moves at  $6.67$  m/s ( $400$  m/min). With 0.1 mm encoder resolution, this generates  $66,700$  pulses per second – requiring dedicated high-speed counter modules (Allen-Bradley 1734-VHSC or Beckhoff EL5101) that operate independently of the PLC scan cycle. [AutomationDirect] (<https://library.automationdirect.com/plc-scan-time-control-system/>) Each bottle pitch spans approximately  $800$  encoder pulses at this resolution, providing ample granularity for position-accurate valve triggering and reject timing.



On rotary fillers, the main carousel encoder serves as the **\*\*master axis\*\***. All other axes — infeed star wheel, discharge star wheel, capper carousel, labeler — are electronically geared or cammed to this master. [Tolomatic] (<https://www.tolomatic.com/blog/demystifying-electronic-camming/>) Krones pioneered this approach with their F1 filler at drinktec 2005, replacing mechanical gearbox synchronization with individual servo motors on each star wheel. Electronic cam profiles are stored as digital position lookup tables (master position → slave position), enabling format changeovers in seconds versus hours for mechanical cam changes. [Tolomatic] (<https://www.tolomatic.com/blog/demystifying-electronic-camming/>)

### ### Bottle pitch control via timing screws and star wheels

Bottles arriving from upstream conveyors in random mass-flow spacing must be separated into precise, single-file arrangement at exact pitch. **\*\*Timing screws\*\*** (feed screws/worm gears) are spiral devices that transition bottles from mass flow into evenly-spaced single file. [Changeparts](<https://www.changeparts.com/star-wheels-and-guides/>) The screw's pitch geometry progressively separates bottles as they advance. **\*\*Star wheels\*\*** are grooved wheels that index containers in and out of fill zones with one bottle per groove pocket. [Liquidpackagingsolution] (<https://www.liquidpackagingsolution.com/news/filling-machine-options---indexing-systems>) On modern Krones systems, each star wheel has its own servo motor (Monotec column design), enabling electronic cam synchronization that replaces the traditional mechanical gear train running beneath the machine table. Missing bottles are detected by photoelectric sensors, triggering "no bottle, no fill" interlocks — the valve simply doesn't open for that position. [Packaging Machinery](<https://taizypackaging.com/automatic-liquid-filling-machine/>) Krones' ErgoBloc L extends this to "no bottle, no label" — if the blow molder rejects a preform, the labeler's vacuum cylinder retracts, wasting no label or glue. [Krones](<https://www.krones.com/en/company/press/magazine/reference/ergobloc-l-quantum-leap-in-block-technology.php>)

### ### Filling valve synchronization and multi-phase profiles

Modern electronic filling uses multi-phase profiles optimized for speed and quality. For water (non-carbonated), the sequence is: **\*\*bottle centering\*\*** via neck-handling grippers → **\*\*pre-fill\*\*** at reduced PFR speed to prevent splash → **\*\*main fill\*\*** at maximum flow rate (up to 250 mL/sec) → **\*\*top-off\*\*** with flow rate reducing as target volume approaches → **\*\*valve close\*\*** triggered by flow meter signal → **\*\*settling phase\*\*** [Krones] (<https://www.krones.com/en/products/machines/modulfill-vfj-water.php>) → **\*\*drip prevention\*\*** via anti-drip sieve in the product discharge → **\*\*bottle discharge\*\*** to outfeed star wheel. For carbonated beverages, additional phases include counter-pressurization, isobaric filling under ~3 bar, and snifting (controlled pressure relief to prevent foaming). [Krones] (<https://www.krones.com/en/products/machines/modulfill-hrs-wine.php>)

### ### Anti-splash engineering for water

Preventing splash at high speed uses several complementary techniques. **\*\*Flow rate profiling\*\*** through PFR valves provides infinitely variable speed — slow start, full flow, slow finish — eliminating the binary fast/slow limitation of conventional two-speed valves. [Krones](<https://www.krones.com/en/products/machines/modulfill-vfs-m-with-pfr-valves.php>) **\*\*Bottom-up filling\*\*** (diving nozzle) inserts the nozzle to the bottle bottom, filling upward as the nozzle retracts with the rising liquid surface, minimizing turbulence. [Gssmachine](<https://www.gssmachine.com/ibc-filling-system/>) **\*\*Swirl inserts\*\*** direct product flow along the bottle wall in a laminar spiral pattern. **\*\*Anti-splash nozzle designs\*\*** include split-tip nozzles (three outlets directing flow down container sides), [Drifton]

(<https://www.drifton.eu/shop/16-accessories-for-pumps/2456-anti-splash-filler-nozzles-for-tubing/>) flow-screen nozzles (mesh producing smooth laminar flow), [Paquip](<https://paquip.com/products/n500-filler-nozzle-assembly>) and inverted cone nozzles (diffused fill pattern). [Volumetrictechnologies](<https://volumetrictechnologies.com/dispensing-nozzles>) Krones' Modulfill HEL specifically targets "minimum oxygen intake and foam creation via the bottom filling method." [Krones](<https://www.krones.com/en/products/machines/modulfill-hel-juice-glas.php>)

### ### CIP integration in control logic

Clean-In-Place systems are fully automated through the PLC with complex valve sequencing. [Wikipedia](<https://en.wikipedia.org/wiki/Clean-in-place>) A standard CIP cycle follows: pre-rinse with hot water (~5 min) → caustic wash (1.5% NaOH at 80°C) → intermediate rinse → acid wash → final rinse (verified by conductivity/pH sensors) → sanitize (steam at 120°C or 1.5% peracetic acid). [IC Filling Systems](<https://www.icfillingsystems.com/cip-cleaning-in-place-filling-machines/>) The control software switches between production and CIP modes via recipe management, where CIP recipes define valve sequences, temperatures, concentrations, flow rates, and durations per circuit. [Cybertrol](<https://www.cybertrol.com/configurable-clean-in-place-cip-solutions-for-end-user-flexibility>) Krones uses automatic CIP cups that swing into place using magnets — contact-free, eliminating pneumatic components. [Krones](<https://www.krones.com/en/products/machines/modulfill-vfs-m-with-pfr-valves.php>) [Krones](<https://www.krones.com/en/products/machines/modulfill-vfs-water.php>) CIP medium drains through open filling valves into a trough with recirculation pump. Monitoring follows the \*\*TACT framework\*\*: Temperature (real-time), Action (chemical concentration via conductivity probes), Contact/flow (minimum 1.5 m/s for turbulent cleaning), and Time (configurable per step). [ScienceDirect](<https://www.sciencedirect.com/science/article/abs/pii/S0956713524005942>) A large-scale reference: Danone's Mizone plant CIP system manages \*\*10 cleaning tanks, 91 circuits, 1,048 pumps/valves, 1,578 sensors, and 188 CIP order types\*\*. [ScienceDirect](<https://www.sciencedirect.com/science/article/abs/pii/S0956713524005942>)

---

## ## 5. C++ implementation for hard real-time bottling control

### ### Memory management: zero allocation after startup

In a hard real-time fill control path executing every 250 µs, dynamic memory allocation is forbidden — `malloc`/`new` have unbounded worst-case execution times and fragment the heap. All memory must be \*\*statically allocated at initialization\*\*. Use `std::array<T, N>` for fixed-size containers. The Embedded Template Library (ETL) provides `etl::vector<T, N>` (fixed-capacity, no heap), `etl::ring\_buffer<T, N>`, and `etl::pool<T, N>` for object pools. For the fill control FIFO tracker, a statically-allocated ring buffer sized to the maximum number of bottles between detection and reject stations eliminates all runtime allocation:

```
```cpp
template<typename T, size_t N>
class StaticRingBuffer {
    std::array<T, N> buffer_;
    std::atomic<size_t> head_{0};
    std::atomic<size_t> tail_{0};
};
```

```
public:
    bool push(const T& item) { /* ... */ }
    bool pop(T& item) { /* ... */ }
};
...
```

Memory pools pre-allocate fixed-size blocks at startup for objects that need create/destroy semantics (e.g., alarm records, batch entries):

```
```cpp
template<typename T, size_t N>
class MemoryPool {
    alignas(T) std::byte storage_[N][sizeof(T)];
    std::bitset<N> used_;
public:
    T* allocate();    // O(N) scan for free slot
    void deallocate(T* p); // O(1) mark free
};
...
```
```

### Lock-free inter-task communication

Between the fill control task (250  $\mu$ s cycle) and the sequence logic task (4 ms cycle), lock-free SPSC (Single-Producer Single-Consumer) queues eliminate mutex contention. The [rigtorp/SPSCQueue](https://github.com/rigtorp/SPSCQueue) [GitHub] (<https://github.com/rigtorp/SPSCQueue>) and [cameron314/readerwriterqueue](https://github.com/cameron314/readerwriterqueue) libraries provide wait-free, fixed-size implementations [GitHub] (<https://github.com/cameron314/readerwriterqueue>) with **50–100 ns** latency between cores. [GitHub] ([https://github.com/MengRao/SPSC\\_Queue](https://github.com/MengRao/SPSC_Queue)) Key implementation details: `alignas(64)` on head/tail indices prevents false sharing between CPU cache lines; [GitHub] (<https://github.com/rigtorp/SPSCQueue>) power-of-2 capacity enables fast modulo via bitwise AND; `std::atomic` with `memory_order_acquire`/`memory_order_release` (not `seq_cst`) minimizes memory barrier overhead. [Hashnode] (<https://radiantsoftware.hashnode.dev/c-lock-free-queue-part-i>)

### Compile-time computation and static polymorphism

`constexpr` eliminates runtime computation for machine parameters:

```
```cpp
constexpr float kBottlesPerMinute = 5000.0f;
constexpr float kCycleTimeMs = 60000.0f / kBottlesPerMinute; // 12.0ms
constexpr uint32_t kEncoderPulsesPerPitch = 800;
constexpr float kRejectDistanceMm = 1500.0f;
constexpr float kPitchMm = 80.0f;
constexpr uint32_t kRejectDelayPulses =
    static_cast<uint32_t>(kRejectDistanceMm / kPitchMm * kEncoderPulsesPerPitch);
```
```

For hot-path sensor drivers where virtual function vtable lookup overhead is unacceptable, CRTP (Curiously Recurring Template Pattern) provides static polymorphism with zero runtime cost: [Cppreference] (<https://en.cppreference.com/w/cpp/language/crtp.html>)

```
```cpp
template<typename Derived>
class SensorDriver {
public:
    int32_t read() { return static_cast<Derived*>(this)->readImpl(); }
};

class FlowMeterDriver : public SensorDriver<FlowMeterDriver> {
    friend class SensorDriver<FlowMeterDriver>;
    int32_t readImpl() { return *reinterpret_cast<volatile uint32_t*>(0x40004000); }
};
```
```

CRTP eliminates the vtable pointer (saving 4/8 bytes per object), removes indirection, and enables full inlining. [GitHub] (<https://gist.github.com/MangaD/0b405df5b91f2ce9cd3fe2cf28c3bc3d>) Use CRTP in the deterministic fill control loop; use virtual dispatch in configuration and startup paths where the overhead is irrelevant. [sqlpey] (<https://www.sqlpey.com/c++/c++-crtp-compile-time-polymorphism/>)

### ### Compiler flags and language standards

**C++17** is the preferred standard for new embedded projects: ``if constexpr`` for compile-time branching, ``std::optional`` for safe value-or-nothing semantics, structured bindings for cleaner code, and ``std::byte`` for type-safe buffer handling. C++11 is the minimum (atomics, ``constexpr``, ``static_assert``, move semantics). Critical compiler flags: ``-fno-exceptions`` (eliminates exception handling overhead; use error codes or ``tl::expected<T, E>`` instead), ``-fno-rtti`` (removes runtime type information), ``-O2`` or ``-Os`` (optimization without aggressive unrolling that bloats instruction cache). For ``volatile`` versus ``std::atomic``: use ``volatile`` exclusively for memory-mapped hardware registers (prevents compiler optimization of reads/writes), and ``std::atomic<T>`` exclusively for inter-thread shared variables (provides memory ordering guarantees). Never mix these use cases.

### ### Static analysis and safety certification

**MISRA C++:2023** — the merged MISRA C++ and AUTOSAR C++14 guideline — is increasingly required for **SIL 2** certification in industrial automation. TÜV-certified analysis tools include Perforce Helix QAC (the gold standard), Parasoft C/C++test, LDRA, and MathWorks Polyspace. Lighter-weight options (PVS-Studio, cppcheck, PC-lint) catch common defects but lack formal certification. For SIL 2, the IEC 61508 safety lifecycle requires documented requirements, code review, static analysis against a recognized coding standard, and coverage testing.

### ### Recommended codebase structure

...

bottling-firmware/

- |— hal/           # Hardware Abstraction Layer
  - | |— gpio.h        # GPIO read/write (volatile register access)
  - | |— timer.h       # Hardware timer configuration
  - | |— ethercat\_hw.h # EtherCAT frame-level interface
- |— bsp/           # Board Support Package
  - | |— board\_config.h # Pin assignments, clock tree, memory map
  - | |— startup.cpp    # System init, watchdog config
- |— drivers/       # Device Drivers (one per physical device type)
  - | |— flow\_meter.h   # E+H Promag electromagnetic flow meter
  - | |— servo\_drive.h  # AX8000 CSP mode interface
  - | |— pneumatic\_valve.h # PFR valve / solenoid valve
  - | |— encoder.h      # Incremental/absolute encoder
  - | |— load\_cell.h    # HBM weighing sensor
- |— middleware/    # Reusable platform services
  - | |— pid\_controller.h # PID with anti-windup, feedforward, cascade
  - | |— packml\_fsm.h    # 17-state PackML state machine
  - | |— fifo\_tracker.h  # Encoder-driven bottle tracking
  - | |— spsc\_queue.h    # Lock-free inter-task queue
  - | |— alarm\_manager.h # Alarm/event system with severity levels
  - | |— recipe\_manager.h # Recipe storage, validation, download
- |— application/    # Application-specific logic
  - | |— fill\_controller.h # Fill sequence, PID tuning, two-speed profile
  - | |— conveyor\_ctrl.h  # Speed control, zone management, indexing
  - | |— reject\_ctrl.h    # Shift-register tracking, air blast timing
  - | |— safety\_iface.h   # Safety PLC communication (FSOE/PROFIsafe)
  - | |— cip\_controller.h  # CIP recipe execution, TACT monitoring
  - | |— hmi\_interface.h  # PackTag exposure via OPC UA
- |— config/         # Compile-time configuration
  - | |— machine\_config.h # constexpr machine parameters
  - | |— recipe\_defaults.h # Default recipe values
- |— test/
  - | |— unit/          # Google Test / Catch2 on x86 host
  - | |— sil/          # Software-in-the-loop with simulated I/O
  - | |— hil/          # Hardware-in-the-loop test harnesses

...

All hardware access flows through the HAL via interface classes (`IFlowMeter``, `IServo``, `IValve``), enabling mock injection for unit tests on the host. Production drivers implement these interfaces with real hardware register access; test mocks provide injectable values. Google Test or Catch2 run PID controller tests, state machine transition tests, and FIFO tracking tests entirely on x86 without target hardware. Hardware-in-the-loop testing connects the real controller to a

simulated plant model (Beckhoff TwinCAT supports this via MATLAB/Simulink integration). Digital twin approaches using Siemens NX MCD or B&R Scene Viewer enable virtual commissioning before physical deployment.

---

## ## 6. Standards compliance: from PackML to FDA to cybersecurity

### ### Functional safety: SIL 2 / PLd as the baseline

Risk assessment per ISO 12100 drives safety requirements. For high-speed rotary fillers with exposed rotating machinery, pneumatic actuators, and pressurized product tanks, most safety functions require **SIL 2 (IEC 62061) / PLd (ISO 13849-1)**, corresponding to a PFHd (Probability of dangerous failure per hour) of  **$10^{-7}$  to  $10^{-6}$** . Access gates to the high-speed carousel may require **SIL 3 / PLc** depending on severity assessment. Required safety functions include: emergency stop (dual-channel, ISO 13850), guard interlocking with guard locking, light curtains at operator access points, and safe motion functions per IEC 61800-5-2 — specifically **STO** (Safe Torque Off) for immediate power removal, **SS1** (Safe Stop 1) for controlled deceleration followed by STO, and **SLS** (Safely Limited Speed) enabling operator access at reduced speed for cleaning or inspection. Safe communication protocols (PROFIsafe, CIP Safety, or FSoE) carry safety data over the standard fieldbus with black-channel principles.

### ### FDA compliance for automated water bottling

21 CFR Part 129 governs processing and bottling of drinking water. Equipment requirements mandate that all plant equipment (tanks, piping, fittings, fillers, cappers, conveyors) must be suitable for intended use, with inspection records maintained. Filling, capping, and sealing must be performed in sanitary conditions precluding contamination. Each package requires a unit production code (date, lot, distribution). Bacteriological testing requires quarterly swab counts ( $\leq 1$  bacteria/mL, coliform-free). **21 CFR Part 11** governs electronic records generated by automated filling equipment: system validation (IQ/OQ/PQ), computer-generated audit trails with timestamps, role-based access control, electronic signatures linked to signed records, and data retention for the entire required period ( $\geq 2$  years per Part 129). ALCOA principles (Attributable, Legible, Contemporaneous, Original, Accurate) apply to all electronic batch records, fill weight data, and quality test results. Material contact surfaces must be **316L stainless steel** with  $Ra \leq 0.8 \mu\text{m}$  per EHEDG guidelines, using FDA 21 CFR 177.2600 compliant gaskets (EPDM, Viton, silicone) and NSF H1 registered food-grade lubricants.

### ### Fill accuracy: legal metrology requirements

The EU e-mark directive (76/211/EEC) defines tolerable negative errors (TNE) by package size. For a **500 mL** water bottle, **TNE = 15 mL** — no bottle may contain less than 470 mL (2× TNE below nominal), and the average across a statistical sample must meet or exceed 500 mL. US NIST Handbook 133 establishes equivalent Maximum Allowable Variation tables. In-line verification uses high-speed checkweighers (Mettler-Toledo, Wipotec, Ishida) operating at 1,200+ packages/min per lane with  $\pm 0.1$  g accuracy, supplemented by vision systems for transparent bottle fill level inspection. Statistical process control using  $\bar{X}$ -R charts with  $Cpk \geq 1.33$  provides continuous process monitoring.

### ### Weihenstephan Standards and vertical integration

Beyond PackML, the beverage industry uses **Weihenstephan Standards (WS Pack, currently WS.10)** developed by the Technical University of Munich with Krones, Sidel, and KHS participation. WS defines **440+ data points** covering machine states, counters, and quality parameters — more granular than PackML for beverage-specific OEE analysis. Communication uses WS OPC UA or the proprietary WS Protocol over TCP/IP. For vertical integration, **OPC UA (IEC 62541)** serves as the standard transport from machine to MES to ERP, with the OPC 30050 companion specification mapping PackML directly to OPC UA information models. ISA-95 / IEC 62264 defines the data exchange model between control systems (Level 2) and manufacturing operations management (Level 3/MES). IEC 62443 industrial cybersecurity applies at **Security Level 2** for typical bottling plants — protection against intentional misuse with simple means — implemented through network segmentation (Purdue Model zones and conduits), PLC/HMI hardening, role-based access control, and OPC UA's built-in X.509 certificate authentication with AES-256 encryption.

---

## ## Conclusion: architecture decisions for production C++ code

The research reveals five critical engineering decisions. First, **target the Beckhoff TwinCAT 3 + EtherCAT platform** for custom C++ development — it provides native TcCOM C++ modules running in hard real-time at 50  $\mu$ s, the fastest fieldbus synchronization ( $\pm 20$  ns EtherCAT DC), and the most natural path from C++ source to deterministic execution. Second, **architect for parallel lines, not a single superhuman machine** — three 100K BPH Krones-class lines with independent controllers coordinated by a supervisory MES layer is how real facilities achieve extreme throughput. Third, **implement the PackML 17-state model as the backbone** of the application layer, exposing PackTags via OPC UA for line integration and OEE calculation. Fourth, **use electromagnetic flow meters with cascade PID control** for water filling — this provides  $\pm 0.25\%$  accuracy at full speed without the mechanical complexity of weight-based systems. Fifth, **enforce the hard real-time C++ discipline** throughout: all memory statically allocated, SPSC lock-free queues between tasks, CRTP for hot-path polymorphism, MISRA C++:2023 compliance for SIL 2 certification, and a clean HAL abstraction enabling host-based unit testing with mock hardware interfaces. The 12 ms per-bottle budget is generous relative to the 50–250  $\mu$ s control cycle times achievable on modern platforms — the real challenge is not raw speed but determinism, fault tolerance, and regulatory compliance across hundreds of concurrent fill operations.